7CCSMIEF Coursework II

K25120204

April 2026

Statement on the Use of Generative AI

Declaration of Use: In accordance with KCL's Model 2 (Authorised use of Generative AI tools to perform literature search), I confirm that Generative AI was used strictly as a search assistant to identify and gather relevant academic literature and conceptual frameworks. I have independently read, verified, and critically appraised all references included. All arguments, interpretations, and the final artefact were constructed solely by the author.
Tools Used: Gemini (Google)
Prompts Given: "Identify and gather relevant literature regarding the use of cubic B-spline embeddings in deep hedging."
Application of Output: The output was used to identify key authors and papers. This provided a foundation for subsequent independent research.

1 Introduction

This essay expands upon the topic presented by Dr. Daniel Mayenberger's lecture on "Cutting-edge AI/ML Methods - Applications in the Financial Industry", specifically the second half of his lecture covering the Optimization of Deep Hedging frameworks using Cubic B-Splines.

Derivatives are financial contracts that derive their value from an underlying asset such as a stock or bond, and are fundamental instruments bought primarily for speculative leverage or to hedge against portfolio risk. In the context of hedging, accurately pricing and risk-managing financial portfolios has remained a well-known problem for decades. In ideal, frictionless markets where trading activity incurs no costs, classical financial mathematics provides elegant solutions in the form of risk neutral pricing. The calculation of "Greeks", such as Delta, alongside the assumption of continuous, cost-free trading, theoretically allows for derivatives to be perfectly replicated and hedged against their underlying assets.

However, real-world trading is affected by proportional trading costs, market impact, and is constrained by finite liquidity, severely limiting the practical application of elegant analytical solutions. To address these frictions, Buehler

et al. introduced "Deep Hedging" [1] , a framework employing deep neural networks to determine optimal discrete strategies to hedge based on quantitative market features. While these neural network implementations successfully optimise hedgers under real-world frictions by maximising risk-adjusted returns, they can require vast parameter inputs to do so, and often produce non-smooth hedging functions.

This essay argues that replacing dense neural network layers with Cubic B-Splines when hedging price-dependent options can drastically reduce parameter counts and guarantee twice-differentiable, continuous solutions whilst still yielding superior P&L in environments with transaction costs.

2 Literature and Theory

2.1 Classical Models and Constraints

Solutions to hedging derivatives rooted in classical financial mathematics, the most famous of which is the Black-Scholes-Merton model, relies on continuous-time stochastic calculus to derive analytical solutions for pricing derivatives and hedging. In this ideal, conceptual market environment, the risk associated with an option can be perfectly eliminated via replication, continuously re-weighting a portfolio based on the derivatives sensitivities, or "Greeks" (e.g., $\Delta = \frac{\partial V}{\partial S}$ in Black-Scholes-Merton). However, these elegant analytical solutions rely on highly restrictive assumptions: a market environment with zero trading costs, continuous trading capabilities, and infinite liquidity resulting in zero market impact.

In reality, participation in real-world financial markets is constrained by frictions, rendering continuous delta-hedging mathematically impossible and financially disastrous. Soner, Shreve, and Cvitanic [2] discuss the limitations of classical mathematical theory when faced with transaction costs in their paper "There is no Nontrivial Hedging Portfolio for Option Pricing with Transaction Costs". Their study investigated the super-replication of a European call option, and found the optimal, cheapest strategy to be trivial: simply buying and holding the underlying asset from the start. As a result, when hedging by replication in real markets, traders are forced to accept a residual hedging error, balancing a risk-reward trade off instead of attempting to perfectly eliminate risk.

2.2 Deep Hedging and New Limitations

In an attempt to navigate the unmanageable pricing and hedging problems in incomplete markets, quantitative finance research has increasingly turned to investigate the application of machine learning methods. Buehler et al. introduced the "Deep Hedging" framework [1], which instead frames this hedging problem as a discrete-time stochastic control problem. Deep Hedging aims to optimise convex risk measures (such as Expected Shortfall in the Deep Hedging

study) over simulated paths by using neural networks to parametrise the hedging strategy. This approach avoids an analytical solution to overcome the parameter dimensionality explosion that comes with attempting to hedge multiple assets using classical methods, allowing for computationally viable discrete-time hedging of complex portfolios whilst incorporating transaction costs and liquidity constraints.

While Deep Hedging provides robust solutions for hedging with market frictions, the use of deep neural networks introduces new challenges in the form of parameter instability and model interpretability. Standard activation functions - most famously ReLU ($max(x, 0)$) - produce hedging strategies that are not naturally smooth, leading to undefined higher-order derivatives and potentially unstable outputs. When handling derivatives in risk management, twice-differentiable pricing functions are critical for modelling curvature risk (like Gamma, $\Gamma = \frac{\partial^2 V}{\partial S^2}$) and ensuring stable hedging strategies. Furthermore, although the study proves neural network solutions satisfy mathematical requirements in replication and equivalent martingale measures (EMM), an architecture utilising a large number of hidden layers creates a "black box" with an uninterpretable parameter and weight count, obscuring output reasoning.

2.3 Cubic B-Splines

To address these problems, this essay proposes the use of Cubic B-Splines within the Deep Hedging framework. As demonstrated by Corlay [3], B-splines are an established, highly efficient method for modelling sufficiently smooth financial curves, such as volatility surfaces. By directly embedding B-splines into neural network layers, it is possible to ensure twice-differentiable outputs within reasonable parameter ranges, achieving the flexibility of machine learning while maintaining the stable outputs required in institutional risk management. As introduced by Dr. Mayenberger [4], B-splines are piecewise polynomial functions defined recursively over a set of "knots" t according to the Cox-de Boor recursion formula:

$$B_{i,k}(x) = \frac{x - t_i}{t_{i+k} - t_i} B_{i,k-1}(x) + \frac{t_{i+k} - x}{t_{i+k} - t_{i+1}} B_{i+1,k-1}(x)$$

where $B_{i,k}$ represents the i-th basis function of order k, and the base case $B_{i,1}(x)$ is a step function equal to 1 if $t_i \leq x < t_{i+1}$ and 0 otherwise.

Dr. Mayenberger presents a further refinement to the approach: whilst the use of a multi-variate spline for all derivative parameters (Moneyness, Time to Expiry, Volatility, Previous Hedge) also results in a combinatorial parameter explosion, this can be avoided by implementing multiple single-dimension splines. A 1-dimensional spline with varying knots can be fitted against each dimension (e.g., 16 knots for moneyness, 4 for each other dimension) and combined with an affine function to produce a solution guaranteed to be sufficiently smooth while keeping parameter counts drastically lower than a neural network.

3 Methodology

To validate the Cubic B-spline approach in an incomplete market, a Monte Carlo simulation was constructed in Python using the NumPy and Pytorch libraries to replicate the optimisation problem presented by Dr. Mayenberger [4]. The experiment mirrors his parameters, modelling the discrete hedging of a European Call option over a 6-month maturity ($T = 0.5$) under Black-Scholes dynamics, with a starting asset price $S_0 = 1.0$, strike $K = 1.0$, and a constant implied volatility $\sigma = 0.20$. The simulation generated 5000 Geometric Brownian Motion (GBM) paths, evaluated at 21 hedging intervals. To simulate real-world market frictions, a transaction cost of $c = 0.5\%$ was applied to all trades. A transaction cost proportional to the dollar value of a trade rather than a fixed cost is an important distinction here, as it explains why the model may avoid rebalancing when the stock price is high even when the hedge adjustment is small.

The baseline strategy followed the analytical Black-Scholes-Merton (BSM), using their formula to derive 'Delta' ($\Delta = \Phi(d_1)$), representing the analytical hedge weighting. The total P&L for this strategy was calculated by subtracting the compounding transaction costs from the replication portfolio's final payoff.

The 'optimised' strategy instead utilised a PyTorch neural network architecture based on B-spline bases. Keeping dimensionality as low as possible, 8 bases were allocated to the Moneyness dimension, and 4 bases to Time-to-Maturity. Following Dr. Mayenberger's methodology, these basis functions were fed into a single affine linear layer with a sigmoid activation, effectively reducing the entire Deep Hedging problem to just 13 tunable parameters. These parameters were optimised using the Adam optimiser, deployed over 300 epochs to minimise the variance of the final P&L distribution.

4 Analysis of Results

4.1 Terminal P&L

The results of the simulation support this essay's proposition and provides insight into the trade-offs within algorithmic hedging: the 13-parameter affine spline successfully learned the core dynamics of option replication without any exposure to the Black-Scholes formula.

Although the empirical P&L distribution seen in Figure 1 demonstrates that the Spline optimisation yielded a higher variance (0.000156) compared to the analytical BSM baseline (0.000112), this outcome remains theoretically sound and highlights the trade-off between optimised approximation and analytical precision.

The BSM baseline possesses complete mathematical "understanding" of the underlying distribution dynamics used in the generation of the GBM paths, allowing for Delta to represent a near-perfect replication under stable volatility. The affine spline, however, attempts to approximate this non-linear relation-

ship using only 13 linear parameters (8 moneyness knots + 4 time knots + 1 bias). The difference in final P&L variance being so marginal represents an impressively close approximation and is a testament to the efficiency of B-splines. Furthermore, the slight disparity in variance should not be viewed solely as an error in approximation. While the BSM baseline is theoretically optimal in its precision in a frictionless environment, it fails to account for the 0.5% in its hedging decisions. The Spline model, however, optimises the variance of the total P&L including transaction costs. The model is able to learn and employ a 'cost-aware' strategy that a rigid analytical approach would not be able to adapt to.

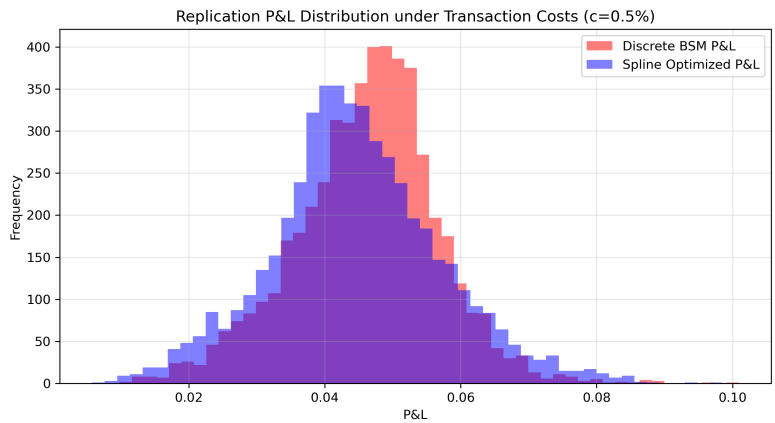

Figure 1: Final P&L comparing analytical BSM discrete hedging against a 13-parameter PyTorch affine spline optimisation.

Moreover, the true benefits to the B-spline approach emerge when considering hedging scenarios beyond the simplified Black-Scholes assumptions. While the analytical BSM Delta dominates in a perfectly log-normal environment (as GBM follows), constrained by a fixed volatility parameter ($\sigma = 0.20$), true volatility is unknown, and real-world hedging is subject to stochastic volatility and jump diffusions, as demonstrated by the Heston model comparison in Dr. Mayenberger's lecture. In these complex environments, hedging only via analytically derived Greeks fails catastrophically. Splines, being empirically optimised on historical/simulated P&L rather than a rigid formula, are far more robust and would represent the only viable method for managing curvature risk in this scenario.

4.2 3D Hedge Surfaces

While the P&L distributions confirm the general viability of the B-spline approach, a critical analysis of the optimised hedging surface (Figure 2) reveals some limitations of the purely additive affine transformation used.

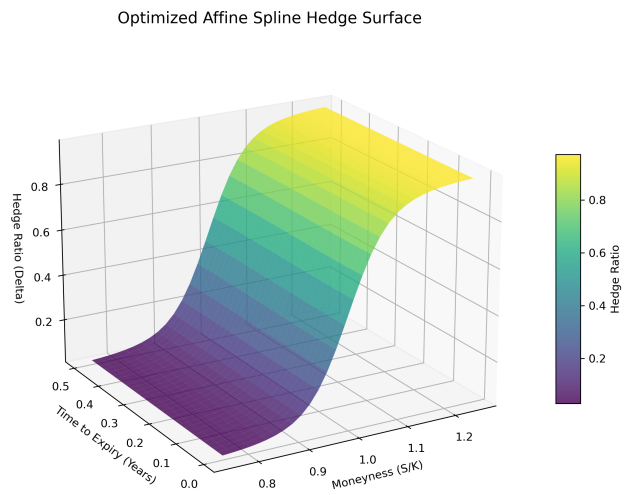

Figure 2: Optimised Affine Spline Hedge Surface

It can be seen that the 3D surface plot correctly models the sigmoidal transition across the moneyness dimension, however, it does so in relatively uniform fashion across the Time-to-Expiry axis. In standard Black-Scholes dynamics, the Gamma ($\Gamma = \frac{\partial^2 V}{\partial S^2}$) of an at-the-money option spikes significantly due to its non-linear relationship. As such, one would expect a replication surface to steepen noticeably at $S/K = 1.0$ as $T \to 0$, as seen in Figure 3

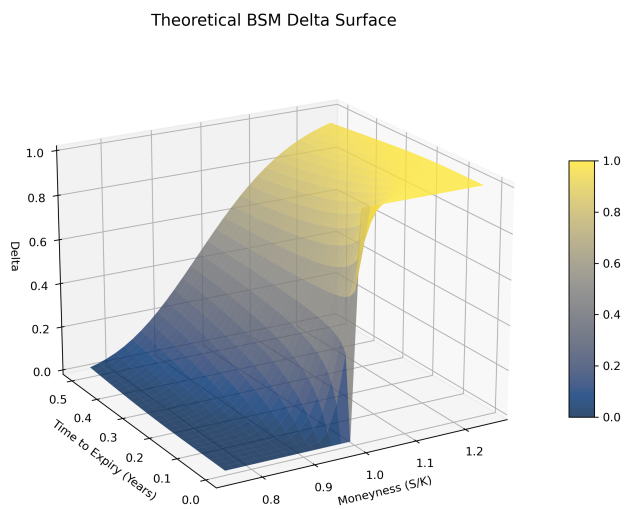

Figure 3: Theoretical BSM Delta Surface for a European Call Option

The observed uniform smoothness in the PyTorch model stems from the

architecture constraints of our low-parameter experiment. By utilising independent 1-dimensional splines and combining them within a single layer ($y = Wx + b$), the model fits an additive relationship between Moneyness and Time. In reality, the sensitivity of Delta to Time is extremely localised and cannot be easily replicated with an additive layer. This empirical observation validates a trade-off often seen in computational finance: while the low dimensional, 13-parameter affine spline successfully prevents over-fitting and maintains interpretability, it lacks the universal approximation capability of a multi-layered deep neural network. To better capture the interaction of Gamma decay under frictions, a further developed model would either require a tensor product of the spline bases instead of a simple concatenation, or a deeper neural network acting on the spline boundaries.

Beyond the architectural limitation, this observed smoothness also reflects an economic rationality to some extent. In an environment with transaction costs, the theoretical Gamma spike of the BSM model around the expiry time becomes a significant liability. Attempting to match a hedge to the BSM Delta's extreme sensitivity as $T \to 0$ would require an immense trading capacity, leading to a catastrophic transaction-cost-induced variance in final P&L. By maintaining a smooth gradient, the Spline model instead learns to under-react to sensitivities. This behaviour is consistent with Leland's 1985 study "Option Pricing and Replication with Transactions Costs" [5], where the optimal hedge under frictions was determined to be a "bounded" version of the frictionless analytical solution, limiting reactions to Greeks sensitivity to avoid bleeding P&L via over-trading.

5 Conclusion

The evolution of derivatives pricing has long been representative of a fine balance between the analytical elegance of the Black-Scholes-Merton model, and the realised behaviour of incomplete markets rooted in reality. While the solutions produced by classical financial mathematics provide precise solutions in frictionless conditions, they become computationally and economically expensive - if not impossible - to replicate when transaction costs are introduced. This essay has demonstrated that whilst modern Deep Hedging offers a robust alternative in real-market conditions, its reliance on densely-layered, high-parameter neural networks can greatly obscure interpretability and produce insufficiently smooth solutions required for institutional risk management.

The empirical results of this study validate the use of a Cubic B-Spline as an architectural improvement to a Deep Hedging approach. Despite reducing the state space to only 13 tunable parameters, the model achieved a replication P&L variance nearly identical to the analytical BSM baseline (0.000156 vs 0.000112). Beyond just simplifying the architecture, the B-spline enforces a twice-differentiable continuous surface, ensuring that the resulting hedge solution is stable, and allowing for the calculating of second-order Greeks such as Gamma and Vanna, otherwise impossible with non-smooth neural network

outputs. In addition, the smooth surface observed along the B-spline's time-to-expiry axis can be viewed as an optimisation response to experiment's induced market frictions. Where the BSM model follows an aggressive rebalancing as time-to-expiry approaches, the spline model learns to follow a conservative approach. This recognises the extreme cost of achieving perfect delta-neutrality, and instead produces an operationally realistic hedge balancing costs with variance reduction.

Looking forward, as discussed by Dr. Mayenberger, future research will likely look to extend these optimisations beyond Geometric Brownian Model to encompass environments where volatility modelling is stochastic, as in real-world markets. Implementations calibrated to approaches such as the Heston model, and optimisation of alternative risk measures such as Expected Shortfall represent logical next steps in capturing improved behaviour. Ultimately, the integration of Cubic B-splines into the Deep Hedging approach suggest a future where algorithmic derivatives hedging follows an intelligent approach to the underlying mechanics as opposed to the size of a neural network.

References

[1] H. Buehler, L. Gonon, J. Teichmann, and B. Wood, "Deep hedging," *SSRN*, 2018. DOI: `https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3120710`.

[2] H. M. Soner, S. E. Shreve, and J. Cvitanic, "There is no nontrivial hedging portfolio for option pricing with transaction costs," *Ann. Appl. Probab.*, 1995. DOI: `https://doi.org/10.1214/aoap/1177004767`.

[3] S. Corlay, "B-spline techniques for volatility modeling," *arXiv*, 2013. DOI: `https://doi.org/10.48550/arXiv.1306.0995`.

[4] D. D. Mayenberger, *Cutting edge ai/ml methods - applications in the financial industry*, 2026. [Online]. Available: `https://keats.kcl.ac.uk/pluginfile.php/12184743/course/section/2209036/2026%20Kings%20College%20-%20AIML%20applications%20in%20banking.pdf`.

[5] H. E. Leland, "Option pricing and replication with transactions costs," *The Journal of Finance*, 1985. DOI: `https://doi.org/10.2307/2328113`.

Appendix A: Code Availability

The Python code developed for this experiment, including the plots generated, is hosted on GitHub. The repository includes the raw simulation parameters and final data visualisation.

- **Repository URL:** https://github.com/DanIsCold/spline-hedging